Reviewer Pack — SOS Moment Matrix Rank and Max-CUT Approximation Ratio

Entry c27792e507e5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 11:29:34 UTC

SOS Moment Matrix Rank and Max-CUT Approximation Ratio

Entry ID: `c27792e507e5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-01 11:29:34 UTC

1. Conjecture

Field A (mathematical branch): Convex Geometry **Field B** (complexity object): Sum-of-Squares Degree of Max-CUT

Statement:

> For every n-vertex Max-CUT instance, the rank of its degree-d SOS moment matrix is at least Ω(n^{1-1/d}) if the approximation ratio is below 0.878 - ε for any ε > 0.

Rationale (proposer's reasoning):

> The rank of the SOS moment matrix encodes the geometry of the feasible region for semidefinite relaxations. High rank implies the relaxation cannot be captured by low-degree polynomials, directly linking geometric complexity to approximation guarantees.

Taxonomy category: `SOS_DEGREE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7f6fcf52135c6485`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_max_cut_instance(n):
    G = [[0] * n for _ in range(n)]
    for i, j in combinations(range(n), 2):
        if random.random() < 0.5:
            G[i][j] = G[j][i] = random.randint(1, 10)
    return G

def degree_d_sos_moment_matrix(G, d):
    n = len(G)
    M = [[0] * (n ** d) for _ in range(n ** d)]
    for i in range(n):
        for j in range(i + 1, n):
            if G[i][j]:
                for k in range(d):
                    M[i * n**(k-1):(i+1)*n**(k-1), j*n**(k-1):(j+1)*n**(k-1)] += [[G[i][j]] * n**(
    return M

def eigenvalue_decomposition(M):
    n = len(M)
    A = [row[:] for row in M]
    for i in range(n):
        A[i][i] -= max(abs(A[j][i]) for j in range(n) if j != i)
    eigenvalues = []
    while len(eigenvalues) < n:
        v = [random.random() for _ in range(n)]
        v /= math.sqrt(sum(x**2 for x in v))
        v_next = [sum(A[i][j] * v[j] for j in range(n)) for i in range(n)]
        v_next /= math.sqrt(sum(x**2 for x in v_next))
        eigenvalues.append(v_next[0])
        for i in range(n):
            A[i][i] -= v_next[i]
    return eigenvalues

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    G = generate_max_cut_instance(n)
    max_cut_ratio = 0.878 - random.random() * 0.01
    conjecture_holds = True
    counterexample = ""
```

```
    for d in [2, 3, 4]:
        M = degree_d_sos_moment_matrix(G, d)
        eigenvalues = eigenvalue_decomposition(M)
        rank = sum(1 for x in eigenvalues if abs(x) > 1e-6)
        required_rank = n ** (1 - 1 / d)

        if rank < required_rank and max_cut_ratio <= 0.878:
            conjecture_holds = False
            counterexample = f"n={n}, d={d}, rank={rank}, required_rank={required_rank}"

    return {
        "metric_name": "Rank",
        "metric_value": rank,
        "instances_tested": n * 3,  # 3 instances per seed for different d
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fracti
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE insufficient data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9699fbd.py", line 82, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9699fbd.py", line 59, in run_trial
    M = degree_d_sos_moment_matrix(G, d)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a9699fbd.py", line 31, in degree_d_sos_
```

```
    M[i * n**(k-1):(i+1)*n**(k-1), j*n**(k-1):(j+1)*n**(k-1)] += [[G[i][j]] * n**(2*k-2)]
     ~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: list indices must be integers or slices, not tuple
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to a TypeError, preventing data collection. Pre-registered support condition cannot be evaluated without successful runs. | next: Fix the TypeError in degree_d_sos_moment_matrix by correcting the list indexing operation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111534
2	propose	ollama_remote	qwen3:8b	0	0	106180
3	preregistration	ollama_remote	qwen3:8b	0	0	24503
4	novelty	ollama_remote	qwen3:8b	0	0	20697
5	novelty	ollama_remote	qwen3:8b	0	0	14891
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22008
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10961
8	judge	ollama_remote	qwen3:8b	0	0	14722

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 325496 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/c27792e507e5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c27792e507e5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c27792e507e5.tar.gz` (if generated)